Mock Midterm For CSC182

Object-Oriented Programming in C++ — Practice Exam

Name: ..... ID: .....

Section: ..... Time: 90 minutes

Question 1: The code contains 10 errors, find and correct them:

```
1   #include <iostream>
2   using namespace std;
3
4   class Circle {
5   private:
6   double radius
7   public:
8   void Circle(double r) {
9   r = radius;
10  }
11  double getArea() {
12  return 3.14 * radius * radius
13  }
14  void setRadius(double r) {
15  radius = r;
16  }
17  Double getRadius() {
18  return Radius;
19  }
20  }
21
22  int main {
23  Circle c(5);
24  c.radius = 10;
25  cout << getArea(c) << endl;
26  return 0;
27  }
```

Question 2: Class Car

1- Create a class named Car with private data members:

- **brand of type string**
- **model of type string**
- **year of type int**
- **price of type double**

2- Implement a parameterized constructor for the class that includes default values:

3- Implement the following public member functions:

- **Setter methods — assign car data**
- **Getter methods — return car data**
- **Print method — prints all car information**

Question 3: Class Garage

1- Create a class named Garage with private data members:

- **car1 of type Car**
- **car2 of type Car**

2- Implement a parameterized constructor for Garage:

3- Implement the following public member functions:

- Print method — prints all Garage information
- Method to return the newer car (the car with the larger year):
- Method that checks if both cars have the same brand:
- Method that returns the total price of both cars:

Question 4: main() Function

1. **Create two Car objects.**
2. **Create a Garage object.**
3. **Print the information of each Car.**
4. **Show which car is newer.**
5. **Check if both cars have the same brand.**
6. **Print the total price of both cars.**

— Good Luck —

Answers

Question 1: The 10 errors and their corrections

#	Line	Wrong	Why	Correct
1	6	`double radius`	Missing semicolon	`double radius;`
2	8	`void Circle(double r)`	A constructor has no return type	`Circle(double r)`
3	9	`r = radius;`	Assignment is reversed (member must receive the parameter)	`radius = r;`
4	12	`return 3.14 * radius * radius`	Missing semicolon	`return 3.14 * radius * radius;`
5	17	`Double getRadius()`	C++ is case-sensitive: the type is `double`	`double getRadius()`
6	18	`return Radius;`	No member named `Radius` (case-sensitive)	`return radius;`
7	20	`}`	A class definition must end with a semicolon	`};`
8	22	`int main {`	Missing parentheses	`int main() {`
9	24	`c.radius = 10;`	`radius` is private → cannot be accessed from main	`c.setRadius(10);`
10	25	`getArea(c)`	`getArea` is a member function → call it on the object	`c.getArea()`

Corrected code

```cpp
#include <iostream>
using namespace std;

class Circle {
private:
    double radius;

public:
    Circle(double r) {
        radius = r;
    }

    double getArea() {
        return 3.14 * radius * radius;
    }

    void setRadius(double r) {
        radius = r;
    }

    double getRadius() {
        return radius;
    }
};

int main() {
    Circle c(5);
    c.setRadius(10);
    cout << c.getArea() << endl;
    return 0;
}
```

Output: `314` (3.14 × 10 × 10)

Question 2: Class Car

1- Class Car with private data members:

```cpp
#include <iostream>
#include <string>
using namespace std;

class Car {
private:
    string brand;
    string model;
    int year;
    double price;
```

2- Parameterized constructor with default values:

```cpp
public:
    // Parameterized constructor with default values
    Car(string b = "Unknown", string m = "Unknown", int y = 2000, double p = 0.0) {
        brand = b;
        model = m;
        year  = y;
        price = p;
    }
```

> Because every parameter has a default value, this one constructor also works as a default constructor: `Car c;` and `Car c("Kia");` are both valid.

3- Setters, getters and print method:

```cpp
    // Setters
    void setBrand(string b) { brand = b; }
    void setModel(string m) { model = m; }
    void setYear(int y)     { year = y; }
    void setPrice(double p) { price = p; }

    // Getters
    string getBrand() { return brand; }
    string getModel() { return model; }
    int    getYear()  { return year; }
    double getPrice() { return price; }

    // Print all car information
    void print() {
        cout << "Brand: " << brand << endl;
        cout << "Model: " << model << endl;
        cout << "Year : " << year  << endl;
        cout << "Price: " << price << endl;
    }
};
```

Question 3: Class Garage

1- Class Garage with private data members:

```
class Garage {
private:
    Car car1;
    Car car2;
```

2- Parameterized constructor:

```
public:
    // Constructor using a member initializer list
    Garage(Car c1, Car c2) : car1(c1), car2(c2) {}
```

> Note: `brand`, `year` and `price` are **private** members of Car, so Garage must use the **getters** (`getYear()`, `getBrand()`, `getPrice()`) — it cannot write `car1.year`.

3- Member functions:

```
    // Print all garage information
    void print() {
        cout << "--- Car 1 ---" << endl;
        car1.print();
        cout << "--- Car 2 ---" << endl;
        car2.print();
    }
```

```
    // Return the newer car (the one with the larger year)
    Car newerCar() {
        if (car1.getYear() > car2.getYear())
            return car1;
        else
            return car2;
    }
```

```
    // Check if both cars have the same brand
    bool sameBrand() {
        if (car1.getBrand() == car2.getBrand())
            return true;
        else
            return false;
    }

    // Return the total price of both cars
    double totalPrice() {
        return car1.getPrice() + car2.getPrice();
    }
};
```

Question 4: main() Function

```cpp
int main() {
    // 1. Create two Car objects (brand, model, year, price)
    Car c1("Toyota", "Camry", 2021, 95000);
    Car c2("Toyota", "Corolla", 2024, 80000);

    // 2. Create a Garage object that holds the two cars
    Garage g(c1, c2);

    // 3. Print the information of each car
    g.print();

    // 4. Show which car is newer
    cout << "\nThe newer car is:" << endl;
    g.newerCar().print();

    // 5. Check if both cars have the same brand
    if (g.sameBrand())
        cout << "\nBoth cars have the same brand." << endl;
    else
        cout << "\nThe cars have different brands." << endl;

    // 6. Print the total price of both cars
    cout << "Total price: " << g.totalPrice() << endl;

    return 0;
}
```

> Common mistake: writing `g.newerCar();` alone prints nothing — the returned Car must be printed: `g.newerCar().print();`

Program output

```
--- Car 1 ---
Brand: Toyota
Model: Camry
Year : 2021
Price: 95000
--- Car 2 ---
Brand: Toyota
Model: Corolla
Year : 2024
Price: 80000

The newer car is:
Brand: Toyota
Model: Corolla
Year : 2024
Price: 80000

Both cars have the same brand.
Total price: 175000
```

Full Program (Questions 2 + 3 + 4 together)

Compiled and tested with g++ — copy it into any compiler (e.g. onlinegdb.com) to run it.

```cpp
#include <iostream>
#include <string>
using namespace std;

// ================= Class Car =================
class Car {
private:
    string brand;
    string model;
    int year;
    double price;

public:
    // Parameterized constructor with default values
    Car(string b = "Unknown", string m = "Unknown", int y = 2000, double p = 0.0) {
        brand = b;
        model = m;
        year  = y;
        price = p;
    }

    // Setters
    void setBrand(string b) { brand = b; }
    void setModel(string m) { model = m; }
    void setYear(int y)     { year = y; }
    void setPrice(double p) { price = p; }

    // Getters
    string getBrand() { return brand; }
    string getModel() { return model; }
    int    getYear()  { return year; }
    double getPrice() { return price; }

    // Print all car information
    void print() {
        cout << "Brand: " << brand << endl;
        cout << "Model: " << model << endl;
        cout << "Year : " << year  << endl;
        cout << "Price: " << price << endl;
    }
};

// ================= Class Garage =================
class Garage {
private:
    Car car1;
    Car car2;

public:
    // Parameterized constructor (member initializer list)
    Garage(Car c1, Car c2) : car1(c1), car2(c2) {}
```

Full Program (continued)

```
53          // Print all garage information
54          void print() {
55              cout << "--- Car 1 ---" << endl;
56              car1.print();
57              cout << "--- Car 2 ---" << endl;
58              car2.print();
59          }
60
61          // Return the newer car (the one with the larger year)
62          Car newerCar() {
63              if (car1.getYear() > car2.getYear())
64                  return car1;
65              else
66                  return car2;
67          }
68
69          // Check if both cars have the same brand
70          bool sameBrand() {
71              if (car1.getBrand() == car2.getBrand())
72                  return true;
73              else
74                  return false;
75          }
76
77          // Return the total price of both cars
78          double totalPrice() {
79              return car1.getPrice() + car2.getPrice();
80          }
81      };
82
83      // ================= main() =================
84      int main() {
85          // 1. Create two Car objects
86          Car c1("Toyota", "Camry", 2021, 95000);
87          Car c2("Toyota", "Corolla", 2024, 80000);
88
89          // 2. Create a Garage object
90          Garage g(c1, c2);
91
92          // 3. Print the information of each car
93          g.print();
94
95          // 4. Show which car is newer
96          cout << "\nThe newer car is:" << endl;
97          g.newerCar().print();
98
99          // 5. Check if both cars have the same brand
100         if (g.sameBrand())
101             cout << "\nBoth cars have the same brand." << endl;
102         else
103             cout << "\nThe cars have different brands." << endl;
104
105         // 6. Print the total price of both cars
106         cout << "Total price: " << g.totalPrice() << endl;
107
108         return 0;
109     }
```